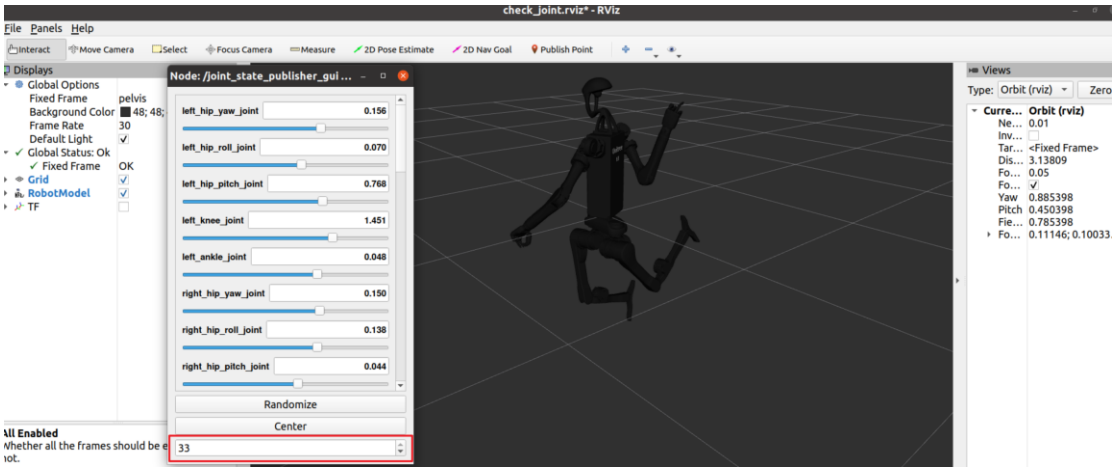
1、 **开源模型分析**：选取一款开源机器人的官方描述包，在 URDF 模型基础上分析其运动学树形结构，分析其包含的自由度数及传递关系，提交分析报告。

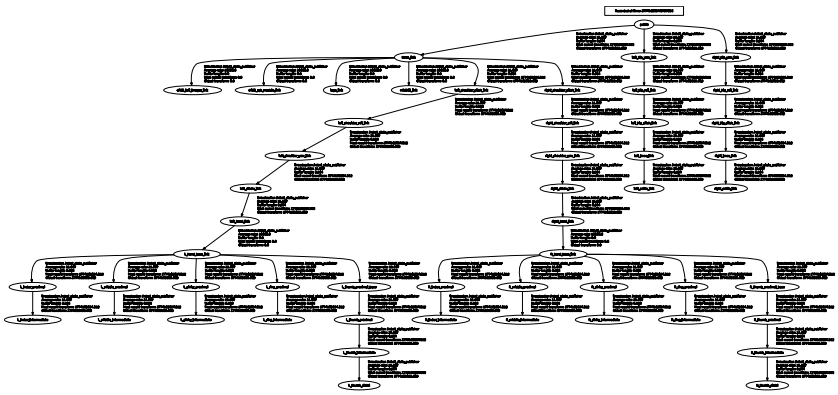
（1）还是选取 Unitree 的官方描述包：h1_with_hand.urdf

（2）按照课上步骤用 roslaunch 和 Rviz 可视化 h1_with_hand.urdf。



可以得知 h1 带灵巧手的人形机器人具有 33 个驱动自由度。再通过如下命令获取运动学树形结构，如下图。

rosrun rqt_tf_tree rqt_tf_tree



（3）自由度数及传递关系分析

传递关系如上图所示，各连杆（椭圆）通过关节（连线）传递。

共计 45 个自由度：

- 驱动自由度（共 33 个）

左右腿（10）：hip_yaw, hip_roll, hip_pitch, knee, ankle

躯干（1）：torso_joint

左右臂（10）：shoulder_pitch, shoulder_roll, shoulder_yaw, elbow, hand_joint

左右手（12）：

大拇指（4）：yaw, proximal_pitch

其他手指（8）：各一个 proximal_joint

- 从动自由度<mimic>（共 12 个）

左右手（12）：

大拇指（4）：intermediate, distal

其他手指（8）：intermediate

2、 **代码编写与对比**：独立手写一个 URDF 文件和一个 Xacro 文件，对比分析两者的实现方式差异，并在两份模型上实现可视化显示。

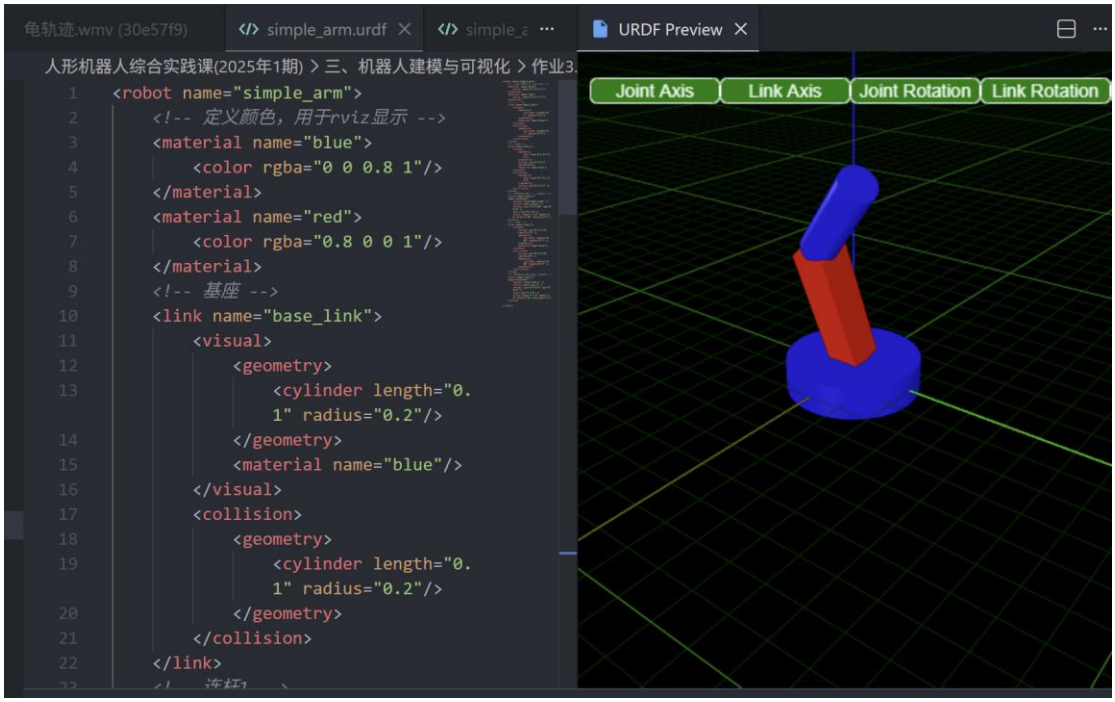
（1）URDF 文件

```
<robot name="simple_arm">
    <!-- 定义颜色，用于rviz 显示 -->
    <material name="blue">
        <color rgba="0 0 0.8 1"/>
    </material>
    <material name="red">
        <color rgba="0.8 0 0 1"/>
    </material>
    <!-- 基座 -->
    <link name="base_link">
        <visual>
            <geometry>
                <cylinder length="0.1" radius="0.2"/>
            </geometry>
            <material name="blue"/>
        </visual>
        <collision>
            <geometry>
                <cylinder length="0.1" radius="0.2"/>
            </geometry>
        </collision>
    </link>
    <!-- 连杆1 -->
    <link name="link_1">
        <visual>
            <geometry>
                <box size="0.1 0.1 0.4"/>
            </geometry>
            <origin xyz="0 0 0.2" rpy="0 0 0"/>
            <material name="red"/>
        </visual>
        <collision>
            <geometry>
                <box size="0.1 0.1 0.4"/>
            </geometry>
            <origin xyz="0 0 0.2" />
        </collision>
    </link>
```

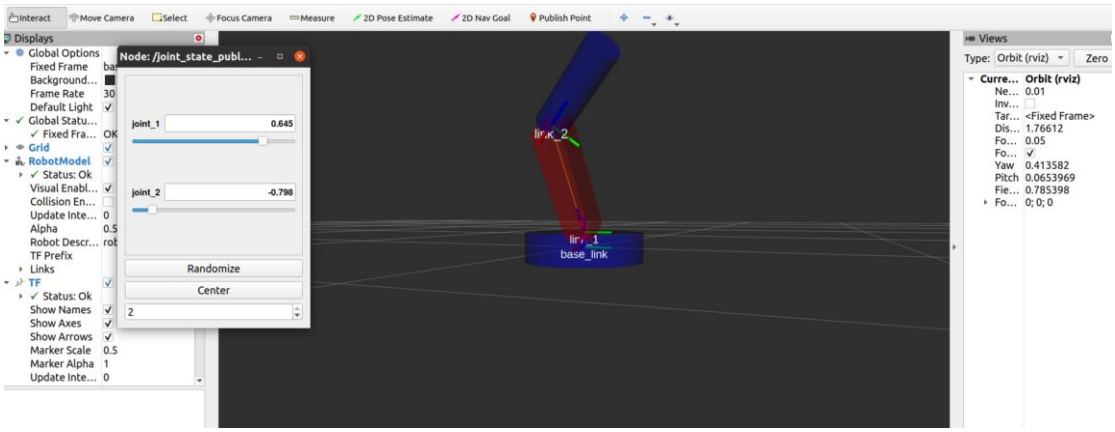
```
    <!-- 节点1（连接基座和连杆1） -->
    <joint name="joint_1" type="revolute">
        <parent link="base_link" />
        <child link="link_1" />
        <origin xyz="0 0 0.05" rpy="0 0 0" />
        <axis xyz="0 1 0" />
        <limit lower="-1.0" upper="1.0" effort="10" velocity="1"/>
    </joint>
    <!-- 连杆2 -->
    <link name="link_2">
        <visual>
            <origin xyz="0 0 0.15" rpy="0 0 0" />
            <geometry>
                <cylinder radius="0.05" length="0.3" />
            </geometry>
            <material name="blue"/>
        </visual>
        <collision>
            <origin xyz="0 0 0.15" rpy="0 0 0" />
            <geometry>
                <cylinder radius="0.05" length="0.3" />
            </geometry>
        </collision>
    </link>
    <!-- 节点2（连接连杆1和连杆2） -->
    <joint name="joint_2" type="revolute">
        <parent link="link_1" />
        <child link="link_2" />
        <origin xyz="0 0 0.4" rpy="0 0 0" />
        <axis xyz="1 0 0" />
        <limit lower="-1.0" upper="1.0" effort="10" velocity="1"/>
    </joint>

</robot>
```

使用 vscode 插件预览 urdf 文件，



执行命令 roslaunch urdf_tutorial display.launch model:=simple_arm.urdf 可视化显示。



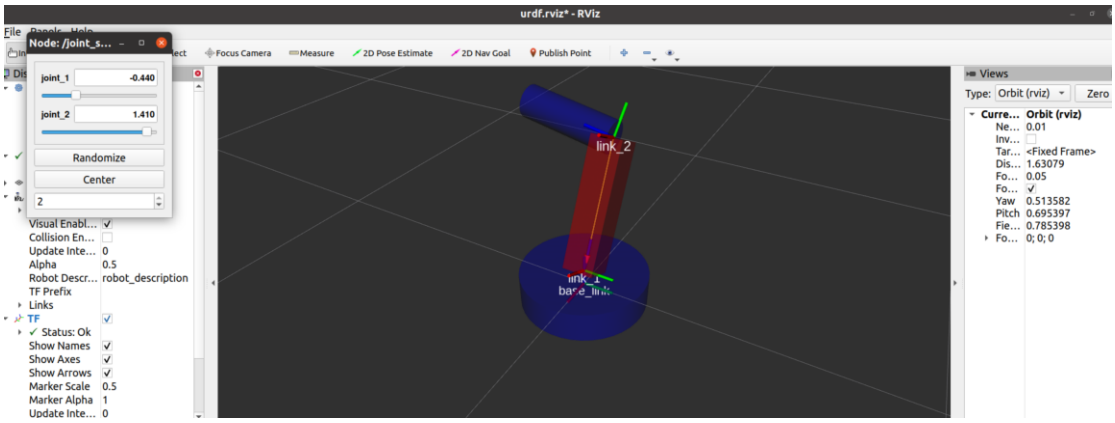
（2）Xacro 文件

```
<robot name="simple_arm_xacro"
  xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:property name="base_length" value="0.1" />
  <xacro:property name="base_radius" value="0.2" />
  <xacro:property name="link1_length" value="0.4" />
  <xacro:property name="link2_length" value="0.3" />

  <!-- 定义颜色，用于rviz 显示 -->
  <material name="blue">
    <color rgba="0 0 0.8 1"/>
  </material>
  <material name="red">
    <color rgba="0.8 0 0 1"/>
  </material>
  <!-- 基座 -->
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="${base_length}" radius="${base_radius}"/>
      </geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <geometry>
        <cylinder length="${base_length}" radius="${base_radius}"/>
      </geometry>
    </collision>
  </link>
  <!-- 连杆1 -->
  <link name="link_1">
    <visual>
      <geometry>
        <box size="0.1 0.1 ${link1_length}"/>
      </geometry>
      <origin xyz="0 0 ${link1_length/2}" rpy="0 0 0"/>
      <material name="red"/>
    </visual>
    <collision>
      <geometry>
        <box size="0.1 0.1 ${link1_length}"/>
      </geometry>
      <origin xyz="0 0 ${link1_length/2}" />
    </collision>
  </link>
  <!-- 节点1（连接基座和连杆1） -->
  <joint name="joint_1" type="revolute">
    <parent link="base_link" />
    <child link="link_1" />
    <origin xyz="0 0 ${base_length/2}" rpy="0 0 0" />
```

```
    <axis xyz="0 1 0" />
    <limit lower="-1.0" upper="1.0" effort="10" velocity="1"/>
  </joint>
  <!-- 连杆 2 -->
  <link name="link_2">
    <visual>
      <origin xyz="0 0 ${link2_length/2}" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.05" length="${link2_length}" />
      </geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <origin xyz="0 0 ${link2_length/2}" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.05" length="${link2_length}" />
      </geometry>
    </collision>
  </link>
  <!-- 节点 2（连接连杆 1 和连杆 2） -->
  <joint name="joint_2" type="revolute">
    <parent link="link_1" />
    <child link="link_2" />
    <origin xyz="0 0 ${link1_length}" rpy="0 0 0" />
    <axis xyz="1 0 0" />
    <limit lower="-${pi/2}" upper="${pi/2}" effort="10" velocity="1"/>
  </joint>
</robot>
```

执行命令 rosrun xacro xacro simple_arm.xacro > simple_arm_xacro.urdf 得到 urdf 文件，再通过 roslaunch 可视化显示。



（3）差异对比：

URDF 文件可以直接用 Rviz 进行可视化，Xacro 文件需要通过 rosrun xacro 处理成 urdf 文件。Xacro 可以理解成将 urdf 中的可变量和重复性的代码抽取出来，使得大型 urdf 的编写、维护更容易。